Cursor AI Code Enhancement Documentation

Project: Movie Finder & Show Booking (Google Colab)

Tool: Cursor AI (Composer)

Date: May 2026

---

1. Original Code

Minimal **ReAct film agent** using Groq (Llama 3.1) and TMDB. Terminal CLI (`input` / `print`) — no GUI, no booking.

```
import os
import re
import json
import requests
from openai import OpenAI
from google.colab import userdata  # Import userdata

os.environ["GROQ_API_KEY"] = userdata.get("GROQ_API_KEY")
os.environ["TMDB_API_KEY"] = userdata.get("TMDB_API_KEY")

client = OpenAI(
    api_key=os.environ["GROQ_API_KEY"],
    base_url="https://api.groq.com/openai/v1",
)

TMDB_KEY = os.environ["TMDB_API_KEY"]
TMDB_BASE = "https://api.themoviedb.org/3"
TMDB_HEADERS = {
    "Authorization": f"Bearer {TMDB_KEY}",  # v4 token
    "accept": "application/json",
}

_YEAR_SUFFIX = re.compile(r"^(?P<title>.+?)\s+(?P<year>(?:19|20)\d{2})$")


def parse_title_and_year(query: str) -> tuple[str, int | None]:
    """If query ends with ' ... 1997', return ('...', 1997); else (stripped query, None)."""
    q = query.strip()
    m = _YEAR_SUFFIX.match(q)
    if not m:
        return q, None
    title = m.group("title").strip()
    year = int(m.group("year"))
    return (title, year) if title else (q, None)


def film_search(film_name: str) -> str:
    try:
        title_q, year_filter = parse_title_and_year(film_name)
        s = requests.get(
            f"{TMDB_BASE}/search/movie",
```

```
                headers=TMDB_HEADERS,
                params={
                    "query": title_q,
                    "include_adult": "false",
                    "language": "en-US",
                    "region": "IN",
                },
                timeout=10,
            ).json()
            if "status_code" in s and s.get("success") is False:
                return f"TMDB auth/error: {s.get('status_message')}"
            results = s.get("results", [])
            if not results:
                return f"No TMDB results for '{title_q}'."
            results = sorted(results, key=lambda r: r.get("popularity", 0), reverse=True)
            if year_filter is not None:
                results = [
                    r
                    for r in results
                    if (r.get("release_date") or "")[:4] == f"{year_filter:04d}"
                ]
                if not results:
                    return f"No TMDB results for '{title_q}' released in {year_filter}."
            results = results[:5]
            blocks = []
            for r in results:
                mid = r["id"]
                details = requests.get(
                    f"{TMDB_BASE}/movie/{mid}",
                    headers=TMDB_HEADERS,
                    params={"append_to_response": "credits", "language": "en-US"},
                    timeout=10,
                ).json()
                crew = details.get("credits", {}).get("crew", [])
                cast = details.get("credits", {}).get("cast", [])
                directors = [c["name"] for c in crew if c.get("job") == "Director"] or ["Unknown"]
                top_cast = [c["name"] for c in cast[:5]] or ["Unknown"]
                blocks.append(
                    f"TITLE: {details.get('title')} ({details.get('original_title')})\n"
                    f"YEAR: {(details.get('release_date') or '????')[:4]}\n"
                    f"LANGUAGE: {details.get('original_language')}\n"
                    f"DIRECTOR: {', '.join(directors)}\n"
                    f"CAST: {', '.join(top_cast)}\n"
                    f"OVERVIEW: {details.get('overview') or ''}"
                )
            return "\n\n=====\n\n".join(blocks)
        except Exception as e:
            return f"TMDB error: {e}"


    TOOLS = {"film_search": film_search}
    TOOL_DESC = """
    Available tools:
    1) film_search(film_name: str) -> str
       Searches TMDB across all languages and includes latest releases.
       Returns up to 5 matching films with director, year, language, and cast.
       If film_name ends with a space and year (e.g. "Dune 2021"), only that release year is returned.
    """.strip()

    # ----------------------------
    # 2) ReAct prompt
    # ----------------------------
    SYSTEM_PROMPT = f"""
    You are a minimal ReAct agent that answers questions about FILMS in any language,
    including the latest releases.

    Internally reason step-by-step using:

    Thought: ...
    Action: film_search OR finish
```

```
Action Input: JSON string, e.g. {{\"film_name\": \"Manjummel Boys\"}}
Observation: tool result (provided by system)

The Observation may contain MULTIPLE candidate films separated by "=====".
Treat each block as a separate film.

When ready, output ONLY this final block:

Final Answer:

For EACH film block in the Observation, output:

--- Match <n>
Title: <title>
Director: <director or Unknown>
Year: <year or Unknown>
Main Cast: <comma-separated actors>
Language: <full language name, e.g. Malayalam, Hindi, Tamil, English>

Rules:
- Always call film_search at least once.
- Do NOT merge different films into one block.
- Roman/English script for names.
- No Thought/Action lines in the final reply.
- If the user gives a year with the title (e.g. "Film 2019"), pass that full string in film_name so the tool can filter
by year.

{TOOL_DESC}
""".strip()


def call_llm(messages, temperature=0):
    resp = client.chat.completions.create(
        model="llama-3.1-8b-instant",
        messages=messages,
        temperature=temperature,
    )
    return resp.choices[0].message.content


def parse_action_block(text: str):
    action_match = re.search(r"Action:\s*(.+)", text)
    action_input_match = re.search(r"Action Input:\s*(.+)", text)
    final_match = re.search(r"Final Answer:\s*(.+)", text, re.DOTALL)
    action = action_match.group(1).strip() if action_match else None
    action_input = action_input_match.group(1).strip() if action_input_match else None
    final_answer = final_match.group(1).strip() if final_match else None
    return action, action_input, final_answer


def run_film_agent(film_name: str, max_steps: int = 5) -> str:
    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": f"Find info about the film: {film_name}"},
    ]

    for _ in range(max_steps):
        assistant_text = call_llm(messages)
        messages.append({"role": "assistant", "content": assistant_text})

        action, action_input, final_answer = parse_action_block(assistant_text)

        if final_answer:
            return f"Final Answer:\n{final_answer.strip()}"

        if not action:
            return "Agent error: no Action found."

        if action.lower() == "finish":
            messages.append({"role": "user", "content": "Provide `Final Answer:` now."})
```

```
                continue

            if action not in TOOLS:
                return f"Agent error: unknown tool `{action}`."

            try:
                parsed = json.loads(action_input)
                if isinstance(parsed, dict):
                    tool_arg = parsed.get("film_name") or parsed.get("query") or str(parsed)
                else:
                    tool_arg = str(parsed)
            except Exception:
                tool_arg = (action_input or "").strip('"')

            observation = TOOLS[action](tool_arg)
            messages.append({"role": "user", "content": f"Observation: {observation}"})

        return "Agent stopped: max steps reached."

    LANG_MAP = {
        "ml": "Malayalam",
        "ta": "Tamil",
        "hi": "Hindi",
        "te": "Telugu",
        "kn": "Kannada",
        "en": "English",
    }


    def lookup_film_direct(query: str, top_n: int = 3) -> str:
        title_q, year_filter = parse_title_and_year(query)
        s = requests.get(
            f"{TMDB_BASE}/search/movie",
            headers=TMDB_HEADERS,
            params={
                "query": title_q,
                "include_adult": "false",
                "language": "en-US",
                "region": "IN",
            },
            timeout=10,
        ).json()
        results = s.get("results", []) or []
        results = sorted(results, key=lambda r: r.get("popularity", 0), reverse=True)
        if year_filter is not None:
            results = [
                r
                for r in results
                if (r.get("release_date") or "")[:4] == f"{year_filter:04d}"
            ]
            if not results:
                return f"No TMDB results for '{title_q}' released in {year_filter}."
        results = results[:top_n]
        if not results:
            return f"No TMDB results for '{title_q}'."

        out = []
        for i, r in enumerate(results, 1):
            d = requests.get(
                f"{TMDB_BASE}/movie/{r['id']}",
                headers=TMDB_HEADERS,
                params={"append_to_response": "credits", "language": "en-US"},
                timeout=10,
            ).json()

            crew = (d.get("credits") or {}).get("crew") or []
            cast = (d.get("credits") or {}).get("cast") or []
            directors = [c["name"] for c in crew if c.get("job") == "Director"]
            top_cast = [c["name"] for c in cast[:6]]
            year = (d.get("release_date") or "????")[:4]
```

```
            out.append(
                f"--- Match {i}\n"
                f"Title: {d.get('title') or d.get('original_title')}\n"
                f"Director: {', '.join(directors) if directors else 'Unknown'}\n"
                f"Year: {year if year != '????' else 'Unknown'}\n"
                f"Main Cast: {', '.join(top_cast) if top_cast else 'Unknown'}\n"
                f"Language: {LANG_MAP.get(d.get('original_language'), d.get('original_language') or 'Unknown')}"
            )
        return "\n\n".join(out)


    def main():
        while True:
            film = input("\nEnter film name (or 'exit'): ").strip()
            if film.lower() == "exit":
                break
            if not film:
                continue
            print(lookup_film_direct(film))


    main()
```

---

2. AI Prompts Used (Cursor AI)

1. Enhance with UI where searched movies contains images like IMDB structure and also add search relevance and add "Did you mean?" for close matches.

2. I also want to include bookmyshow feature on this code. Add BookMyShow-type UI to book films currently running at theatres.

3. Show all movies as poster images; booking (theatre, seats) only after clicking a poster.

4. Show only poster images for currently running movies.

5. Show warning under Confirm Booking if no seats selected; auto-scroll to Book Tickets on poster click; clear previous confirmation.

6. Rename tabs: Movie Info (search movie data) and Book Shows (book currently running shows).

---

3. Enhanced Code

Two-tab **ipywidgets** Colab app: **Movie Info** (search & movie details) and **Book Shows** (now playing posters + demo booking).

```
import os
import re
import json
import html as html_module
import random
```

```
import string
from difflib import SequenceMatcher
from urllib.parse import quote

import requests
from openai import OpenAI
import ipywidgets as widgets
from IPython.display import display, HTML, clear_output, Javascript
from google.colab import userdata, output

os.environ["GROQ_API_KEY"] = userdata.get("GROQ_API_KEY")
os.environ["TMDB_API_KEY"] = userdata.get("TMDB_API_KEY")

client = OpenAI(
    api_key=os.environ["GROQ_API_KEY"],
    base_url="https://api.groq.com/openai/v1",
)

TMDB_KEY = os.environ["TMDB_API_KEY"]
TMDB_BASE = "https://api.themoviedb.org/3"
TMDB_HEADERS = {
    "Authorization": f"Bearer {TMDB_KEY}",
    "accept": "application/json",
}

LANG_MAP = {
    "ml": "Malayalam", "ta": "Tamil", "hi": "Hindi",
    "te": "Telugu", "kn": "Kannada", "en": "English",
    "ja": "Japanese", "ko": "Korean", "fr": "French",
    "es": "Spanish", "de": "German",
}
POSTER_BASE = "https://image.tmdb.org/t/p/w300"
POSTER_BASE_LARGE = "https://image.tmdb.org/t/p/w500"

CITIES = {
    "Mumbai": "mumbai",
    "Delhi-NCR": "national-capital-region-ncr",
    "Bengaluru": "bengaluru",
    "Hyderabad": "hyderabad",
    "Chennai": "chennai",
    "Kolkata": "kolkata",
    "Kochi": "kochi",
    "Pune": "pune",
    "Ahmedabad": "ahmedabad",
    "Chandigarh": "chandigarh",
    "Jaipur": "jaipur",
    "Lucknow": "lucknow",
    "Goa": "goa",
    "Thrissur": "thrissur",
    "Thiruvananthapuram": "trivandrum",
}

MOCK_THEATRES = {
    "Mumbai": ["PVR Phoenix Mills", "INOX R City Mall", "Cinepolis Andheri"],
    "Delhi-NCR": ["PVR Select Citywalk", "INOX Nehru Place", "Carnival Odeon"],
    "Bengaluru": ["PVR Orion Mall", "INOX Garuda Mall", "Cinepolis Forum"],
    "Hyderabad": ["PVR Inorbit", "AMB Cinemas", "Cinepolis Manjeera"],
    "Chennai": ["PVR Ampa Skywalk", "INOX Prozone", "SPI Palazzo"],
    "Kolkata": ["INOX South City", "PVR Quest", "Carnival Lindsay"],
    "Kochi": ["PVR Lulu Mall", "Cinepolis Centre Square", "Q Cinemas"],
    "Pune": ["INOX Bund Garden", "PVR Phoenix", "Cinepolis Seasons"],
    "Ahmedabad": ["PVR Acropolis", "Cinepolis Ahmedabad One", "INOX Himalaya"],
    "Chandigarh": ["PVR Elante", "INOX Piccadily", "Cinepolis DLF"],
    "Jaipur": ["INOX World Trade Park", "PVR Mall of Jaipur", "Cinepolis MI Road"],
    "Lucknow": ["PVR Phoenix Palassio", "INOX Saharaganj", "Cinepolis Emerald"],
    "Goa": ["INOX Panaji", "PVR Mall de Goa", "Cinepolis Fatorda"],
    "Thrissur": ["Kairali Sree", "Ramdas Theatre", "Gokulam Movies"],
    "Thiruvananthapuram": ["Kairali Theatre", "Sree Padmanabha", "Aries Plex"],
}
```

```
MOCK_SHOWTIMES = ["10:00 AM", "1:15 PM", "4:30 PM", "7:00 PM", "10:15 PM"]
MOCK_ROWS = list("ABCDEFGH")
MOCK_SEATS_PER_ROW = 10

NOW_PLAYING_CACHE = []
SELECTED_MOVIE = None
SELECTED_MOVIE_ID = None


def _normalize_title(s: str) -> str:
    s = (s or "").lower().strip()
    s = re.sub(r"[^\w\s]", " ", s)
    return re.sub(r"\s+", " ", s).strip()


def title_relevance(query: str, title: str, original_title: str = "") -> float:
    q = _normalize_title(query)
    if not q:
        return 0.0
    best = 0.0
    for raw in (title, original_title):
        t = _normalize_title(raw)
        if not t:
            continue
        if q == t:
            return 1.0
        if q in t or t in q:
            best = max(best, 0.92)
        q_words = set(q.split())
        t_words = set(t.split())
        if q_words and q_words <= t_words:
            best = max(best, 0.88)
        best = max(best, SequenceMatcher(None, q, t).ratio())
    return best


def did_you_mean_html(query: str, title: str, relevance: float) -> str:
    if relevance >= 0.98 or relevance < 0.55:
        return ""
    if _normalize_title(query) == _normalize_title(title):
        return ""
    return (
        '<div style="margin:0 0 12px 0;padding:10px 14px;border-radius:10px;'
        'background:#fff8e6;border:1px solid #f0d78c;color:#5c4a00;'
        'font-family:system-ui;font-size:14px;">'
        f'Did you mean <b>{title}</b>? '
        f'<span style="color:#7a6a30;font-size:12px;">'
        f'(closest match to &ldquo;{query}&rdquo;)</span></div>'
    )


def bms_book_url(city_name: str, movie_title: str) -> str:
    slug = CITIES.get(city_name, "mumbai")
    return f"https://in.bookmyshow.com/explore/movies-{slug}?query={quote(movie_title)}"


def fetch_now_playing_all(max_pages: int = 10):
    movies = []
    seen_ids = set()
    for page in range(1, max_pages + 1):
        data = requests.get(
            f"{TMDB_BASE}/movie/now_playing",
            headers=TMDB_HEADERS,
            params={"region": "IN", "language": "en-US", "page": page},
            timeout=10,
        ).json()
        results = data.get("results") or []
        if not results:
            break
        for r in results:
```

```
                if not r.get("poster_path"):
                    continue
                mid = r.get("id")
                if mid in seen_ids:
                    continue
                seen_ids.add(mid)
                movies.append({
                    "id": mid,
                    "title": r.get("title") or r.get("original_title") or "Unknown",
                    "poster": POSTER_BASE_LARGE + r["poster_path"],
                    "rating": r.get("vote_average") or 0,
                    "overview": (r.get("overview") or "")[:120],
                    "release_date": (r.get("release_date") or "")[:10],
                    "language": LANG_MAP.get(r.get("original_language"), "—"),
                })
            if page >= (data.get("total_pages") or 1):
                break
        return movies


    def generate_mock_seats():
        seats = []
        for row in MOCK_ROWS:
            for num in range(1, MOCK_SEATS_PER_ROW + 1):
                if random.random() > 0.35:
                    seats.append(f"{row}{num}")
        return sorted(seats)


    def mock_booking_id():
        return "BMS" + "".join(random.choices(string.digits, k=8))


    def _selected_movie_title():
        return SELECTED_MOVIE["title"] if SELECTED_MOVIE else ""


    def render_now_playing_grid(movies, city: str, selected_id=None) -> str:
        movies = [m for m in movies if m.get("poster")]
        if not movies:
            return (
                '<div style="padding:20px;color:#666;font-family:system-ui;">'
                'No poster images for currently running movies.</div>'
            )
        tiles = []
        for m in movies:
            mid = m["id"]
            is_sel = selected_id is not None and mid == selected_id
            border = (
                "outline:3px solid #ffeb3b;outline-offset:3px;"
                if is_sel else "outline:2px solid transparent;"
            )
            title = html_module.escape(m["title"])
            scale_out = "scale(1.02)" if is_sel else "scale(1)"
            if is_sel:
                border = (
                    "outline:3px solid #ffeb3b;outline-offset:3px;"
                    "box-shadow:0 0 12px rgba(255,235,59,0.45);"
                )
            tiles.append(
                f'<div role="button" tabindex="0" title="{title}" '
                f"onclick=\"google.colab.kernel.invokeFunction('on_poster_click', [{mid}], {{}})\" "
                f'style="cursor:pointer;{border}border-radius:8px;overflow:hidden;'
                f'transition:transform 0.15s;line-height:0;" '
                f"onmouseover=\"this.style.transform='scale(1.05)'\" "
                f"onmouseout=\"this.style.transform='{scale_out}'\">"
                f'<img src="{m["poster"]}" alt="{title}" '
                f'style="width:100%;height:220px;object-fit:cover;display:block;" />'
                f"</div>"
            )
```

```
    hint = (
        "Click a movie poster to book tickets (hover to see title)."
        if not selected_id
        else "Selected — scroll down for theatre &amp;amp; seat selection."
    )
    return f"""
    <div style="font-family:system-ui;">
      <div style="background:linear-gradient(90deg,#c4242e,#e23744);color:#fff;
                  padding:14px 18px;border-radius:10px 10px 0 0;font-size:16px;font-weight:700;">
        Now Showing in <u>{html_module.escape(city)}</u>
        <span style="font-weight:400;font-size:13px;"> ({len(movies)} posters)</span>
      </div>
      <div style="background:#1a1a2e;padding:18px;border-radius:0 0 10px 10px;
                  display:grid;grid-template-columns:repeat(auto-fill,minmax(130px,1fr));
                  gap:14px;max-height:560px;overflow-y:auto;">
        PLACEHOLDER_TILES
      </div>
      <div style="margin-top:10px;padding:10px 14px;background:#fff8e6;
                  border:1px solid #f0d78c;border-radius:8px;font-size:12px;color:#5c4a00;">
        {hint}
      </div>
    </div>
    """.replace("PLACEHOLDER_TILES", "".join(tiles))


def rerender_poster_grid():
    with book_output:
        clear_output()
        display(HTML(render_now_playing_grid(
            NOW_PLAYING_CACHE, city_dropdown.value, SELECTED_MOVIE_ID,
        )))


def clear_confirmed_booking():
    with book_confirm_output:
        clear_output()


def scroll_to_book_tickets():
    display(Javascript("""
    setTimeout(function() {
      var el = document.getElementById('book-tickets-section');
      if (el) el.scrollIntoView({behavior: 'smooth', block: 'start'});
    }, 350);
    """))


def clear_seat_warning():
    seat_warning.value = ""


def show_seat_warning():
    seat_warning.value = (
        '<div style="font-family:system-ui;padding:10px 14px;margin:6px 0 0 0;'
        'background:#fff3f3;border:1px solid #f5c6c6;border-radius:8px;color:#a00;'
        'font-size:13px;">'
        '<b>Seats are not selected.</b> Please select at least one seat.</div>'
    )


def hide_booking_panel():
    booking_panel.layout.display = "none"
    book_section_title.layout.display = "none"
    selected_movie_banner.value = ""
    clear_seat_warning()


def show_booking_panel():
    if not SELECTED_MOVIE:
```

```
            hide_booking_panel()
            return
        m = SELECTED_MOVIE
        rating = f'★ {m["rating"]:.1f}' if m.get("rating") else ""
        selected_movie_banner.value = (
            f'<div style="font-family:system-ui;padding:12px 14px;background:#fce8ea;'
            f'border-left:4px solid #e23744;border-radius:8px;margin:12px 0;">'
            f'<div style="font-weight:700;color:#c4242e;font-size:16px;">'
            f'{html_module.escape(m["title"])}</div>'
            f'<div style="font-size:13px;color:#555;margin-top:4px;">'
            f'{rating} · {html_module.escape(m.get("language", ""))} · '
            f'{html_module.escape(m.get("release_date", ""))}</div></div>'
        )
        book_section_title.layout.display = ""
        booking_panel.layout.display = "flex"
        clear_seat_warning()
        refresh_theatres()


    def on_poster_click(movie_id):
        global SELECTED_MOVIE, SELECTED_MOVIE_ID
        movie_id = int(movie_id)
        SELECTED_MOVIE_ID = movie_id
        SELECTED_MOVIE = next((m for m in NOW_PLAYING_CACHE if m["id"] == movie_id), None)
        if SELECTED_MOVIE:
            clear_confirmed_booking()
            clear_seat_warning()
            show_booking_panel()
            rerender_poster_grid()
            scroll_to_book_tickets()
            book_status.value = (
                f'<span style="color:#0a7;">Selected <b>{html_module.escape(SELECTED_MOVIE["title"])}</b> '
                f'— choose theatre &amp;amp; seats below.</span>'
            )
        return movie_id


    output.register_callback("on_poster_click", on_poster_click)


    def search_movies_raw(query: str, top_n: int = 5, min_relevance: float = 0.55):
        s = requests.get(
            f"{TMDB_BASE}/search/movie",
            headers=TMDB_HEADERS,
            params={
                "query": query,
                "include_adult": "false",
                "language": "en-US",
                "region": "IN",
            },
            timeout=10,
        ).json()
        results = s.get("results", []) or []
        scored = []
        for r in results:
            rel = title_relevance(
                query, r.get("title") or "", r.get("original_title") or "",
            )
            if rel >= min_relevance:
                scored.append((rel, r.get("popularity", 0), r))
        suggestion = {"title": None, "relevance": 0.0}
        if scored:
            best_rel, _, best_r = scored[0]
            suggestion = {
                "title": best_r.get("title") or best_r.get("original_title") or "",
                "relevance": best_rel,
            }
        if not scored:
            return [], suggestion
        scored.sort(key=lambda x: (-x[0], -x[1]))
```

```
            results = [r for _, _, r in scored[:top_n]]
            cards = []
            for r in results:
                d = requests.get(
                    f"{TMDB_BASE}/movie/{r['id']}",
                    headers=TMDB_HEADERS,
                    params={"append_to_response": "credits", "language": "en-US"},
                    timeout=10,
                ).json()
                crew = (d.get("credits") or {}).get("crew") or []
                cast = (d.get("credits") or {}).get("cast") or []
                cards.append({
                    "title": d.get("title") or d.get("original_title") or "Unknown",
                    "original_title": d.get("original_title") or "",
                    "year": (d.get("release_date") or "")[:4] or "Unknown",
                    "directors": [c["name"] for c in crew if c.get("job") == "Director"] or ["Unknown"],
                    "cast": [c["name"] for c in cast[:6]] or ["Unknown"],
                    "language": LANG_MAP.get(
                        d.get("original_language"),
                        (d.get("original_language") or "Unknown").title(),
                    ),
                    "rating": d.get("vote_average") or 0,
                    "votes": d.get("vote_count") or 0,
                    "overview": d.get("overview") or "",
                    "poster": (POSTER_BASE + d["poster_path"]) if d.get("poster_path") else None,
                    "tmdb_url": f"https://www.themoviedb.org/movie/{d.get('id')}",
                })
            return cards, suggestion


        def render_cards_html(cards, query: str) -> str:
            if not cards:
                return (
                    '<div style="padding:16px;border-radius:10px;background:#fff3f3;'
                    f'color:#a00;font-family:system-ui;">'
                    f'No close matches for <b>{query}</b>.</div>'
                )
            html = ['<div style="display:flex;flex-direction:column;gap:14px;font-family:system-ui;">']
            for c in cards:
                poster_html = (
                    f'<img src="{c["poster"]}" style="width:120px;height:180px;'
                    f'object-fit:cover;border-radius:8px;flex-shrink:0;" />'
                    if c["poster"] else
                    '<div style="width:120px;height:180px;background:#222;color:#aaa;'
                    'display:flex;align-items:center;justify-content:center;'
                    'border-radius:8px;font-size:12px;">No poster</div>'
                )
                rating = f'⭐ {c["rating"]:.1f} ({c["votes"]} votes)' if c["rating"] else ''
                html.append(f"""
                <div style="display:flex;gap:14px;padding:14px;border:1px solid #e5e7eb;
                            border-radius:12px;background:#fafafa;">
                  {poster_html}
                  <div style="flex:1;min-width:0;">
                    <div style="font-size:18px;font-weight:600;color:#111;">
                      {c["title"]} <span style="color:#666;font-weight:400;">({c["year"]})</span>
                    </div>
                    <div style="color:#666;font-size:13px;margin-bottom:8px;">
                      {c["original_title"]} · {c["language"]} · {rating}
                    </div>
                    <div style="font-size:14px;color:#222;margin:4px 0;">
                      <b>Director:</b> {", ".join(c["directors"])}
                    </div>
                    <div style="font-size:14px;color:#222;margin:4px 0;">
                      <b>Main Cast:</b> {", ".join(c["cast"])}
                    </div>
                    <div style="font-size:13px;color:#444;margin-top:8px;line-height:1.4;">
                      {c["overview"]}
                    </div>
                    <a href="{c["tmdb_url"]}" target="_blank"
                       style="font-size:12px;color:#2563eb;">View on TMDB →</a>
```

```
            </div>
          </div>
          """)
      html.append("</div>")
      return "".join(html)


  def render_booking_confirm(city, movie, theatre, showtime, seats, booking_id, bms_url):
      seat_str = ", ".join(seats) if seats else "—"
      total = len(seats) * 250
      return f"""
      <div style="font-family:system-ui;max-width:480px;">
        <div style="background:linear-gradient(135deg,#c4242e,#e23744);color:#fff;
                    padding:20px;border-radius:12px 12px 0 0;text-align:center;">
          <div style="font-size:28px;">✓</div>
          <div style="font-size:20px;font-weight:700;">Booking Confirmed (Demo)</div>
          <div style="font-size:13px;opacity:0.9;margin-top:4px;">ID: {booking_id}</div>
        </div>
        <div style="border:1px solid #e5e7eb;border-top:none;padding:18px;
                    border-radius:0 0 12px 12px;background:#fafafa;">
          <div style="font-size:16px;font-weight:600;margin-bottom:12px;">{html_module.escape(movie)}</div>
          <div style="font-size:13px;color:#444;line-height:1.8;">
            <b>City:</b> {html_module.escape(city)}<br>
            <b>Theatre:</b> {html_module.escape(theatre)}<br>
            <b>Showtime:</b> {html_module.escape(showtime)}<br>
            <b>Seats:</b> {html_module.escape(seat_str)}<br>
            <b>Amount (demo):</b> ₹{total}
          </div>
          <div style="margin-top:14px;padding:10px;background:#fff3cd;border-radius:8px;
                      font-size:12px;color:#856404;">
            This is a <b>demo booking</b> only. For real tickets, book on BookMyShow.
          </div>
        </div>
      </div>
      """


  def show_suggestion(query: str, suggestion: dict) -> str:
      title = suggestion.get("title")
      rel = suggestion.get("relevance", 0.0)
      hint = did_you_mean_html(query, title or "", rel)
      if hint and title:
          suggest_btn.description = f'Search "{title}" instead'
          suggest_btn.suggestion_title = title
          suggest_btn.layout.display = ""
          suggest_row.layout.display = ""
          return hint
      suggest_btn.layout.display = "none"
      suggest_row.layout.display = "none"
      return ""


  def refresh_theatres(_=None):
      if not SELECTED_MOVIE:
          return
      city = city_dropdown.value
      theatres = MOCK_THEATRES.get(city, MOCK_THEATRES["Mumbai"])
      book_theatre_dropdown.options = theatres
      if theatres:
          book_theatre_dropdown.value = theatres[0]
      refresh_showtimes()


  def refresh_showtimes(_=None):
      book_showtime_dropdown.options = MOCK_SHOWTIMES
      if MOCK_SHOWTIMES:
          book_showtime_dropdown.value = MOCK_SHOWTIMES[0]
      refresh_seats()
```

```
        def refresh_seats(_=None):
            book_seats.options = generate_mock_seats()
            book_seats.value = ()


        def on_load_now_playing(_=None):
            global NOW_PLAYING_CACHE, SELECTED_MOVIE, SELECTED_MOVIE_ID
            city = city_dropdown.value
            book_status.value = f'<span style="color:#555;">Loading all movies in <b>{city}</b>…</span>'
            try:
                SELECTED_MOVIE = None
                SELECTED_MOVIE_ID = None
                hide_booking_panel()
                with book_confirm_output:
                    clear_output()
                NOW_PLAYING_CACHE = fetch_now_playing_all()
                book_status.value = (
                    f'<span style="color:#0a7;">{len(NOW_PLAYING_CACHE)} movies — '
                    f'click a poster to book.</span>'
                )
                rerender_poster_grid()
            except Exception as e:
                book_status.value = f'<span style="color:#a00;">Error: {e}</span>'


        def on_back_to_movies(_=None):
            global SELECTED_MOVIE, SELECTED_MOVIE_ID
            SELECTED_MOVIE = None
            SELECTED_MOVIE_ID = None
            hide_booking_panel()
            with book_confirm_output:
                clear_output()
            book_status.value = '<span style="color:#555;">Click a poster to select a movie.</span>'
            rerender_poster_grid()


        def on_confirm_booking(_=None):
            movie = _selected_movie_title()
            city = city_dropdown.value
            theatre = book_theatre_dropdown.value or "—"
            showtime = book_showtime_dropdown.value or "—"
            seats = list(book_seats.value or [])
            if not movie:
                book_status.value = '<span style="color:#a00;">Click a movie poster first.</span>'
                return
            if not seats:
                show_seat_warning()
                with book_confirm_output:
                    clear_output()
                return
            clear_seat_warning()
            bid = mock_booking_id()
            url = (SELECTED_MOVIE or {}).get("bms_url") or bms_book_url(city, movie)
            book_status.value = f'<span style="color:#0a7;">Demo booking <b>{bid}</b> created.</span>'
            with book_confirm_output:
                clear_output()
                display(HTML(render_booking_confirm(city, movie, theatre, showtime, seats, bid, url)))


        def on_open_bms(_=None):
            movie = _selected_movie_title()
            if not movie:
                book_status.value = '<span style="color:#a00;">Click a movie poster first.</span>'
                return
            url = bms_book_url(city_dropdown.value, movie)
            display(Javascript(f"window.open({json.dumps(url)}, '_blank');"))


        def on_city_change(_=None):
            global SELECTED_MOVIE, SELECTED_MOVIE_ID
```

```
            SELECTED_MOVIE = None
            SELECTED_MOVIE_ID = None
            hide_booking_panel()
            if NOW_PLAYING_CACHE:
                rerender_poster_grid()
                book_status.value = (
                    f'<span style="color:#555;">City: <b>{city_dropdown.value}</b> — click a poster.</span>'
                )


        search_box = widgets.Text(
            placeholder="Search a movie name for cast, plot, ratings…",
            layout=widgets.Layout(width="60%", height="38px"),
        )
        search_btn = widgets.Button(description="Search", button_style="primary", icon="search")
        clear_btn = widgets.Button(description="Clear", icon="trash")
        suggest_btn = widgets.Button(
            description="", button_style="warning", icon="lightbulb-o",
            layout=widgets.Layout(display="none"),
        )
        suggest_row = widgets.HBox([suggest_btn], layout=widgets.Layout(display="none"))
        status = widgets.HTML(value="")
        search_output = widgets.Output()


        def on_search(_=None):
            query = search_box.value.strip()
            if not query:
                status.value = '<span style="color:#a00;">Please type a movie name.</span>'
                return
            status.value = f'<span style="color:#555;">Searching <b>{query}</b>…</span>'
            with search_output:
                clear_output()
            try:
                cards, suggestion = search_movies_raw(query, top_n=5)
                hint = show_suggestion(query, suggestion)
                if not cards:
                    status.value = f'<span style="color:#a00;">No close matches for <b>{query}</b>.</span>'
                    with search_output:
                        if hint:
                            display(HTML(hint))
                    return
                status.value = f'<span style="color:#0a7;">Found {len(cards)} result(s) for <b>{query}</b>.</span>'
                with search_output:
                    if hint:
                        display(HTML(hint))
                    display(HTML(render_cards_html(cards, query)))
            except Exception as e:
                status.value = f'<span style="color:#a00;">Error: {e}</span>'


        def on_clear(_=None):
            search_box.value = ""
            status.value = ""
            suggest_btn.layout.display = "none"
            suggest_row.layout.display = "none"
            with search_output:
                clear_output()


        def on_suggest_click(_=None):
            search_box.value = suggest_btn.suggestion_title
            on_search()


        search_btn.on_click(on_search)
        clear_btn.on_click(on_clear)
        search_box.on_submit(on_search)
        suggest_btn.on_click(on_suggest_click)
```

```
search_tab = widgets.VBox([
    widgets.HTML(
        "<div style='font-family:system-ui;color:#333;margin-bottom:4px;font-size:15px;"
        "font-weight:600;'>Search movies &amp; view details</div>"
        "<div style='color:#555;font-family:system-ui;margin-bottom:8px;font-size:13px;'>"
        "Find cast, director, language, rating, plot summary, and TMDB link. "
        "English, Hindi, Tamil, Malayalam, and more."
        "</div>"
    ),
    widgets.HBox([search_box, search_btn, clear_btn]),
    status, suggest_row, search_output,
])

city_dropdown = widgets.Dropdown(
    options=list(CITIES.keys()), value="Mumbai", description="City:",
    style={"description_width": "40px"}, layout=widgets.Layout(width="280px"),
)
load_now_btn = widgets.Button(
    description="Load running shows", button_style="danger", icon="refresh",
    layout=widgets.Layout(margin="0 0 0 8px"),
)
book_theatre_dropdown = widgets.Dropdown(
    description="Theatre:", options=[], style={"description_width": "55px"},
    layout=widgets.Layout(width="100%"),
)
book_showtime_dropdown = widgets.Dropdown(
    description="Show:", options=MOCK_SHOWTIMES, style={"description_width": "55px"},
    layout=widgets.Layout(width="100%"),
)
book_seats = widgets.SelectMultiple(
    description="Seats:", options=[], rows=8, style={"description_width": "55px"},
    layout=widgets.Layout(width="100%"),
)
confirm_book_btn = widgets.Button(
    description="Confirm Booking", button_style="success", icon="check",
)
seat_warning = widgets.HTML(value="", layout=widgets.Layout(margin="4px 0 0 0"))
# open_bms_btn = widgets.Button(
#     description="Book on BookMyShow", button_style="danger", icon="external-link",
# )
back_btn = widgets.Button(description="← Back to movies", button_style="info", icon="arrow-left")
book_status = widgets.HTML(value="")
book_output = widgets.Output(layout=widgets.Layout(margin="12px 0"))
selected_movie_banner = widgets.HTML(value="")
book_section_title = widgets.HTML(
    value=(
        "<div id='book-tickets-section' style='font-family:system-ui;font-weight:600;color:#e23744;"
        "margin:16px 0 8px 0;font-size:15px;'>Book tickets for this show</div>"
    ),
    layout=widgets.Layout(display="none"),
)
booking_panel = widgets.VBox(
    [
        selected_movie_banner,
        book_theatre_dropdown,
        book_showtime_dropdown,
        book_seats,
        widgets.HBox([confirm_book_btn, back_btn]),
        seat_warning,
    ],
    layout=widgets.Layout(display="none"),
)
book_confirm_output = widgets.Output()


def on_seats_change(_=None):
    if book_seats.value:
        clear_seat_warning()
```

```
load_now_btn.on_click(on_load_now_playing)
city_dropdown.observe(on_city_change, names="value")
book_theatre_dropdown.observe(lambda c: refresh_showtimes(), names="value")
book_showtime_dropdown.observe(lambda c: refresh_seats(), names="value")
book_seats.observe(on_seats_change, names="value")
confirm_book_btn.on_click(on_confirm_booking)
#open_bms_btn.on_click(on_open_bms)
back_btn.on_click(on_back_to_movies)

book_tab = widgets.VBox([
    widgets.HTML(
        "<div style='font-family:system-ui;color:#333;margin-bottom:4px;font-size:15px;"
        "font-weight:600;'>Book currently running shows</div>"
        "<div style='color:#555;font-family:system-ui;margin-bottom:10px;font-size:13px;'>"
        "Pick your city and load now-playing posters. Click a film to choose "
        "theatre, showtime, and seats (demo booking)."
        "</div>"
    ),
    widgets.HBox([city_dropdown, load_now_btn]),
    book_status,
    book_output,
    book_section_title,
    booking_panel,
    book_confirm_output,
])

tabs = widgets.Tab(children=[search_tab, book_tab])
tabs.set_title(0, "Movie Info")
tabs.set_title(1, "Book Shows")

display(widgets.VBox([
    widgets.HTML(
        "<h2 style='font-family:system-ui;margin:0 0 4px 0;'>"
        "Movie Finder &amp;amp; Show Booking</h2>"
        "<div style='color:#888;font-family:system-ui;font-size:13px;margin-bottom:12px;'>"
        "<b>Movie Info</b> — search any film for cast, plot, and ratings (TMDB). "
        "<b>Book Shows</b> — browse films running in theatres near you and book tickets."
        "</div>"
    ),
    tabs,
]))

on_load_now_playing()
```

---

4. Productivity Observation

> Using Cursor AI turned a CLI ReAct notebook into a full Colab UI with tabs, HTML cards, and a demo booking flow in much less time than manual widget and callback wiring. Cursor AI converted a ~250-line CLI ReAct notebook into a ~750-line Colab UI (tabs, HTML cards, booking flow) much faster than hand-writing widgets, callbacks, and layout. Most effort shifted from boilerplate to natural-language requirements, though Colab integration details still needed human review.

---

5. Comparison

Aspect	Original	Enhanced
Interface	Terminal	ipywidgets + HTML
Search UX	Plain text	Cards + relevance + suggestions
Booking	None	Demo BookMyShow-style flow
~Lines	250	750